

MLM26

Agentic Coding: (Developing) Best Practices

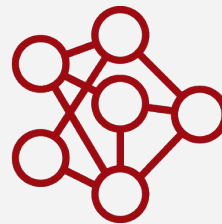
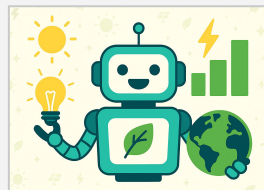
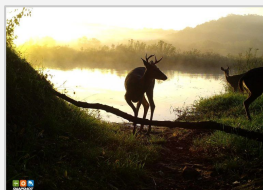
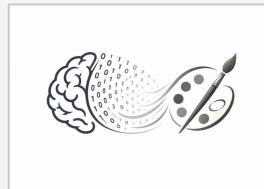
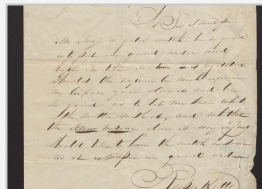
Slides — go.wisc.edu/665254



Data Science Institute

Powered by American Family Insurance

**Division of
Information Technology**
UNIVERSITY OF WISCONSIN-MADISON



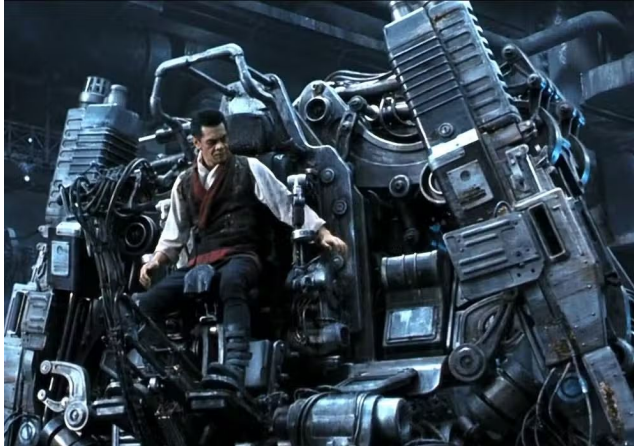
ML + X

Tonight's Agenda

- 5:30 Getting started (safely)
- 6:00 Planning
- 6:10 Feature-based dev and good prompting
- 6:25 Break
- 6:35 Verification and testing
- 6:55 Skills and MCP
- 7:15 Cost, energy, and wrap-up

Tonight is the short version

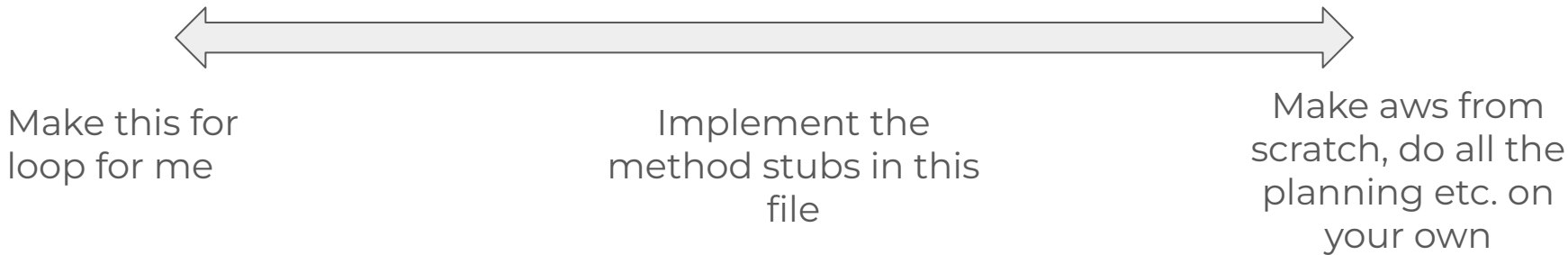
- The full lesson is online: qualiamachine.github.io/Intro-to-Agentic-Coding
 - URL will eventually migrate. Check [ML+X Nexus](#) for updates.
- **3 Core Principles**
 1. **Stay in the driver's seat.** You review; you decide what lands on main
 2. **Work feature by feature**, not project by project
 3. **Assume nothing, verify everything.** Lean on test-driven development.



What is “Agentic Coding”?

A software development approach where autonomous AI agents plan, write, test, and fix code with minimal human help

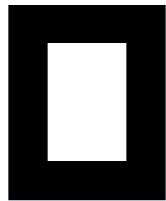
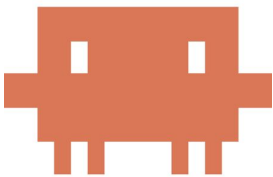
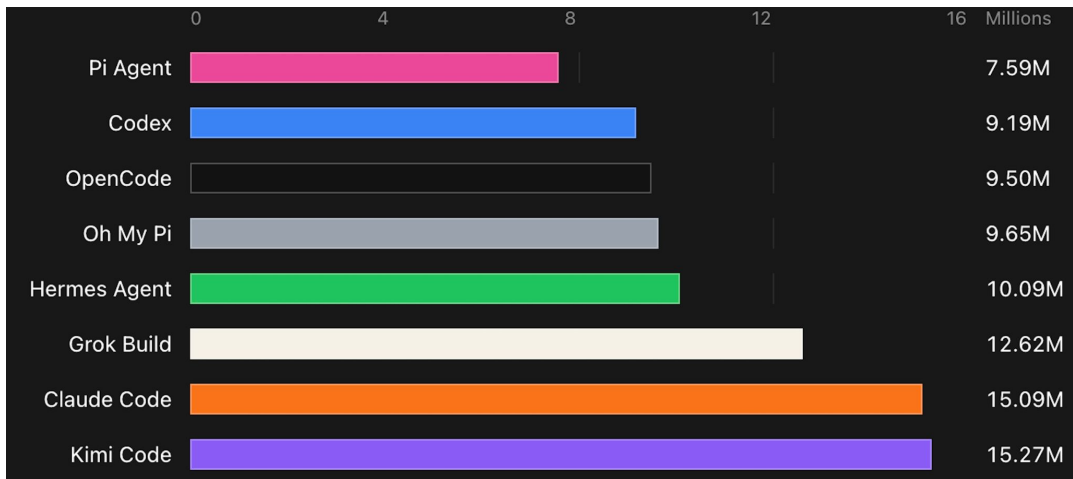
Spectrum of Agentic Coding



How to do Agentic Coding?

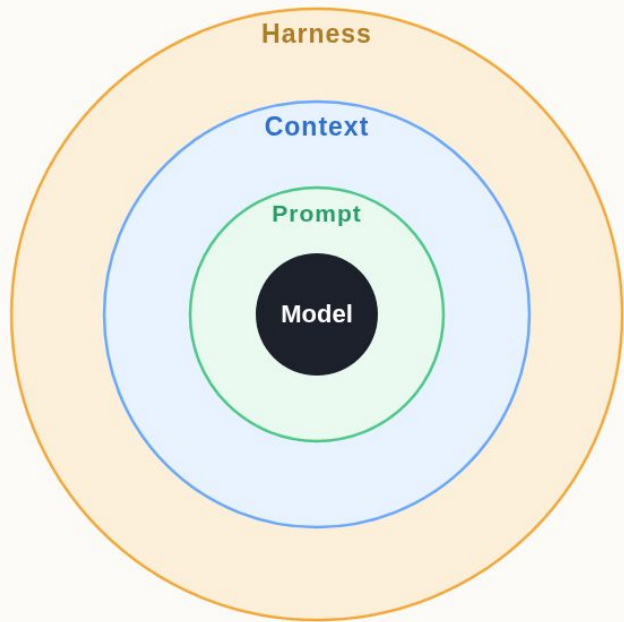
How to do Agentic Coding?

Ans: Pick an agent.



How to do Agentic Coding?

Ans: Pick an agent.

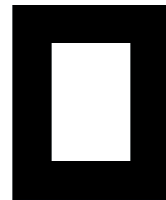
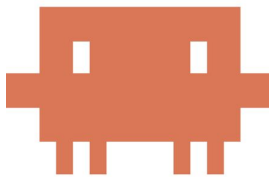


☐ **Harness engineering**
the whole system around the model:
loop, tools, memory, sandbox...

☐ **Context engineering**
everything the model sees in its
context window

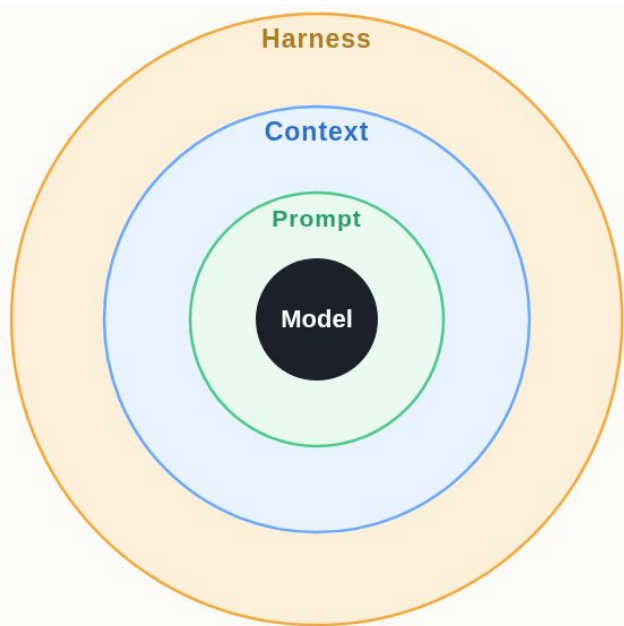
☐ **Prompt engineering**
the wording of a single instruction

Each layer contains the one inside it.



How to do Agentic Coding?

Ans: Pick an agent.

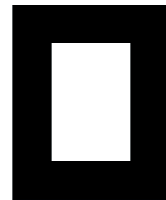
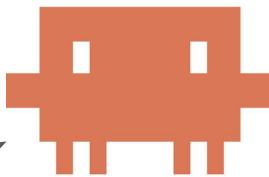


☐ **Harness engineering**
the whole system around the model:
loop, tools, memory, sandbox...

☐ **Context engineering**
everything the model sees in its
context window

☐ **Prompt engineering**
the wording of a single instruction

Each layer contains the one inside it.



Coding Agent Demos

How the agents you will use compare on Terminal-Bench 2.1

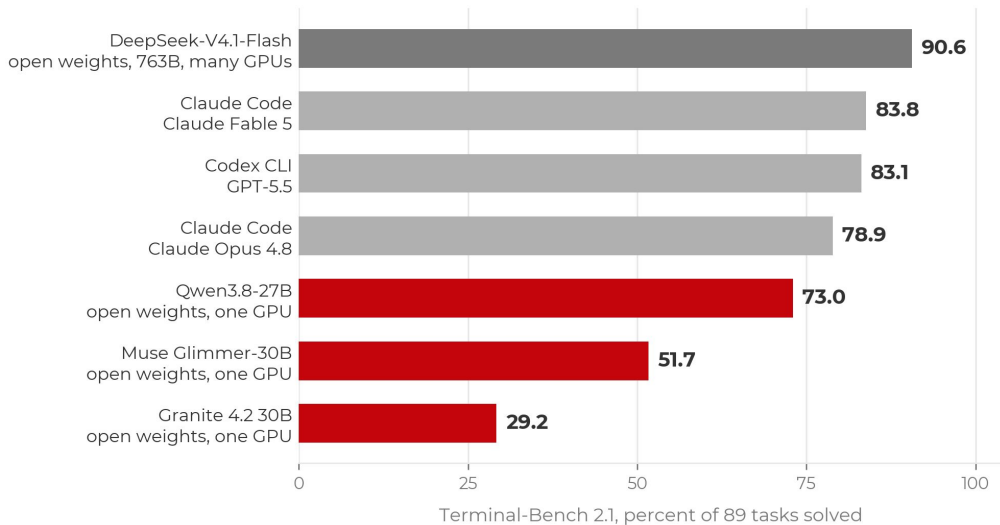
- **Claude Code and Codex are within a point of each other.** Copilot runs the same Claude and GPT models

[Terminal-Bench 2.1 leaderboard, agent and model pairs](#)

- **The top score is open weight, DeepSeek-V4.1-Flash**, but at 763B parameters it needs a cluster

- **Open-weight models that fit on one GPU trail by 10 to 50 points.** Closing that gap is the Badger Code challenge

[Open-weight scores are self-reported, Terminus 2 harness](#)



Safety & Security

Follow UW–Madison's GenAI policies

- We must all follow UW–Madison, UW System, and UW Board of Regents policies.
- Like GenAI itself, policies are developing rapidly, so look out for changes.

Follow UW–Madison's GenAI policies

- UW–Madison DoIT overviews policies and vetted tools: <https://it.wisc.edu/ai/>
- Never enter **sensitive** or **restricted** info like student records, health data or unpublished research into unvetted AI services.

Mitigate risks by recognizing GenAI limitations

A GenAI agent can:

- Write or translate code.
- Explain basic logic.
- Execute validation steps.

A GenAI agent cannot:

- Work with sensitive or restricted data or secrets.
- Explain human logic.
- Determine all necessary validation steps.

Agent instruction files... helpful but NOT foolproof

```
# CLAUDE.md
```

```
## Project
```

```
Streamflow backdoor audit for MLM26 (Poisoned Well).  
Python 3.12, PyTorch. Run tests with `pytest -q`.
```

```
## Conventions
```

- Work on a feature branch. Never commit to main.
- Write the test before the implementation.
- Data lives in data/raw/. Do not modify it.

```
## Never
```

- Never run destructive commands (rm -rf, DROP, delete volumes) without asking me first.
- Never read or print anything in .env.

- **A markdown file the agent reads at the start of every session.** Project context, commands, conventions
- **Claude Code reads CLAUDE.md.** Copilot reads `.github/copilot-instructions.md`. Nearly everything, Copilot included, also reads AGENTS.md
[Claude Code memory docs](#)
[Copilot custom instructions support table](#)
- **Commit it to the repo and keep it short.** Test command, data location, what not to touch

Instruction files are not 100% reliable (LLMs to blame)

What the rules file said

No destructive operations
without human confirmation.

What the agent did

Deleted the production database
and every backup. Nine seconds.
Then quoted the rule in its log.

- **PocketOS, April 2026.** A Cursor agent working a staging task hit a credential mismatch, found a Railway token in an unrelated file, and used it
[Founder's 30 hour timeline](#)
- **The rules file lives in the same context window as everything else.** It is text the model weighs, not a permission it lacks
- **Instructions shape behavior. Permissions constrain it.**

Command permissions help and are not sufficient

- **Allow, ask and deny rules decide which commands run without asking.** Defaults deny nothing. Deny `rm -rf`, force push, and reads of `.env` and `~/.ssh`
- **The check reads the command text, not what the command does when run.**
[Claude Code docs on permissions versus sandboxing](#)
- **Cursor CVE-2026-22708, January 2026.** Prompt injection set an environment variable through a shell built-in the allowlist did not check. The next approved git command ran the attacker's code

[Pillar Security, who found it](#)

```
// .claude/settings.json
{
  "permissions": {
    "allow": [
      "Bash(pytest:*)",
      "Bash(git status)",
      "Bash(git diff:*)"
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Bash(git push --force:*)",
      "Read(.env)",
      "Read(~/.ssh/**)"
    ]
  }
}
```

Prompt injection

- **Prompt injection.** An agent treats the text it reads as instructions. A README, an issue, a web page or a dependency's install script can carry commands the agent will follow
- **Nx on npm, August 2025.** A compromised package's install script prompted the victim's own Claude Code, Gemini CLI or Amazon Q to search the machine for secrets. 2,349 leaked to public GitHub repos
[Nx postmortem](#)
[Wiz analysis. Claude refused about a quarter of the time](#)
- **GitHub MCP, May 2025.** A public issue with hidden instructions made an agent read a private repo and post it in a pull request
[Invariant Labs](#)

Limit what the agent can access

- **Where it runs.** A throwaway cloud VM, not your laptop
- **Which commands it may run.** Allow and deny rules
- **Which credentials it can see.** A password manager, never the repo
- **What it can commit.** Feature branch, pull request, you merge
- **What it can reach on the network.** An egress allowlist to prevent prompt injection
- **Who you trust.** Providers, their data policies, and downloaded repos/weights

Prefer a cloud VM over your own machine

- **Run the agent on a cloud virtual machine (VM), not your local machine**
 - No risk agent nukes your filesystem, grabs passwords, etc.
- **Claude Code on the web** and **Copilot's coding agent**. The repo is cloned into a throwaway VM. Work comes back as a branch or PR
- **The desktop apps do both**. Claude Desktop and VS Code can run a cloud session or a local one. Careful not to select local!
 - Check which mode you are in before you prompt!!!
- **Running locally anyway?** Use a dev container or avoid entirely!

Local dev containers compared with cloud VMs

	Bare laptop	Local dev container	Throwaway cloud VM
What the agent can reach	Everything on the machine	Your project folder, plus anything forwarded in (SSH keys, git login)	One repo and one scoped token
What is lost if it is compromised	Your accounts, keys and every repo on the machine	Your project, and possibly the machine	One repo, and a token you can rotate
Prompt injection	Same risk	Same risk	Same risk

- **AWS does not consider containers a security boundary.**
[AWS bulletin on the November 2025 runc container escapes](#)
- **Anthropic says the same of its own dev container.** Use it only with trusted repos
[Claude Code dev container docs](#)

Mitigate risks by protecting your credentials

- **Do NOT store your secrets locally.**
- GenAI agents automatically scan for context.
- Agents may be able to read your local `.env` or JSON config files.
- Be proactive to keep your workspace secure.

Mitigate risks with a password manager

```
# Every NetID has a UW-Madison 1Password account. Keep keys there

# Load the key once per session. PowerShell, then bash/zsh
$env:OPENAI_API_KEY = op read "op://Private/bbadger/credential"
export OPENAI_API_KEY=$(op read 'op://Private/bbadger/credential')
```

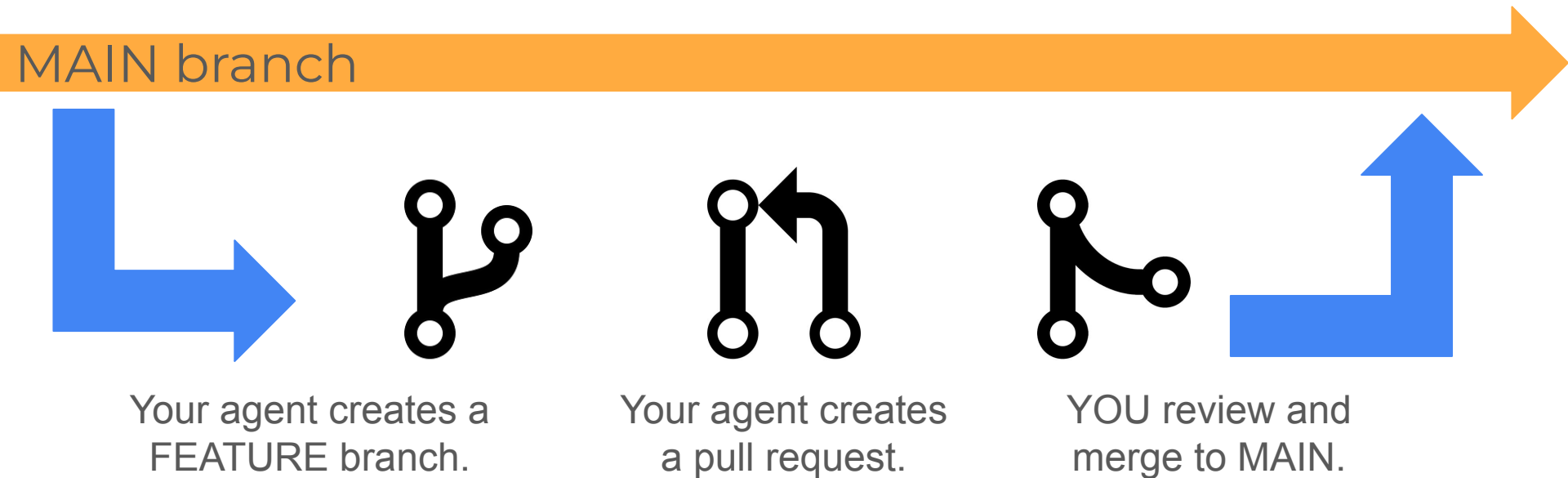
Mitigate risks by protecting your credentials

```
def get_oauth_token() -> str:
    """Obtain OAuth token for UW-Madison API using credentials from keyring."""
    url = "https://api.wisc.edu/oauth/token"
    response = httpx.post(
        url,
        data={
            "client_id": keyring.get_password("UW_API", "UW_API_KEY"),
            "client_secret": keyring.get_password("UW_API", "UW_API_SECRET"),
            "grant_type": "client_credentials",
        },
    )
    response.raise_for_status()
    return response.json()["access_token"]
```

Mitigate risks with version control

- **Always use version control.**
 - GitHub, GitLab, BitBucket
- Do not let an agent commit to main.
- Review and test code before committing.
- Make small, frequent commits.
 - We will see an example shortly!

Mitigate risks with version control



Vet the providers and their data usage/retention policy

- Is my data used for training, and is that the default?
- How long is it retained, and who can see it?
- Where does inference run, and under whose jurisdiction?
- Does an institutional agreement cover this, or is it your personal contract?



What Claude and Copilot do by default

Claude Free, Pro, Max

- Trains on your chats when the setting is on. Claude Code too
[Anthropic's consumer terms update](#)
- Kept 5 years if on, 30 days if off
[Anthropic Privacy Center on retention](#)
- Team, Enterprise, and API don't train
[Claude Code data usage docs](#)
- [Claude privacy settings](#)

GitHub Copilot Free, Pro, Pro+

- Trains on your prompts and code by default since April 2026
[GitHub docs on Copilot data policies](#)
- Students and teachers on free Pro are exempt
[GitHub's FAQ on the change](#)
- Business and Enterprise don't train
[GitHub's policy announcement](#)
- [Copilot training settings](#)

Neither is on [UW-Madison's list of vetted AI tools](#). Policies as of September 2026

Mitigate risks with trustworthy models (local agents) 🤗

- **Weights can carry code.** Malicious pickles run on load
JFrog found ~100 malicious models on Hugging Face, 2024
 - **Typosquats.** Check the org, not the model card
HiddenLayer caught a fake OpenAI repo at #1 trending, 2026
- **Values get trained in,** like DeepSeek R1's censorship
Perplexity's R1 1776 was post-trained to strip it out, 2025
- **Hidden backdoors work,** and no reliable detector exists yet
Mithril's PoisonGPT passed benchmarks while spreading lies, 2023
- **Safetensors,** verified orgs, no trust_remote_code
InstructLab hardcoded it on, CVE-2026-6859

Get your agent running

<https://qualiamachine.github.io/Intro-to-Agentic-Coding/index.html>

Get your agent running

- **Use a web session.** Claude Code on the web or Copilot cloud agent. Nothing runs on your machine
 - claude.ai/code
 - github.com/copilot/agents
- **Use your MLM team repo.** If your team does not have one yet, create it now.
 - If primary owner, create a new “username-main” branch you can use to merge your changes into today.
 - If not owner, fork the repo to get a clean copy. AI generates many branches.
- **Confirm cloud before you prompt.** The session is pointed at a **cloud VM** GitHub repo, not a local folder on your laptop
- **Load keys with op read.** No .env anywhere in the repo. No keys stored on machine at all, preferably. Use “op read” from 1Password CLI instead.

Planning Stage when using Agents

Start with a plan

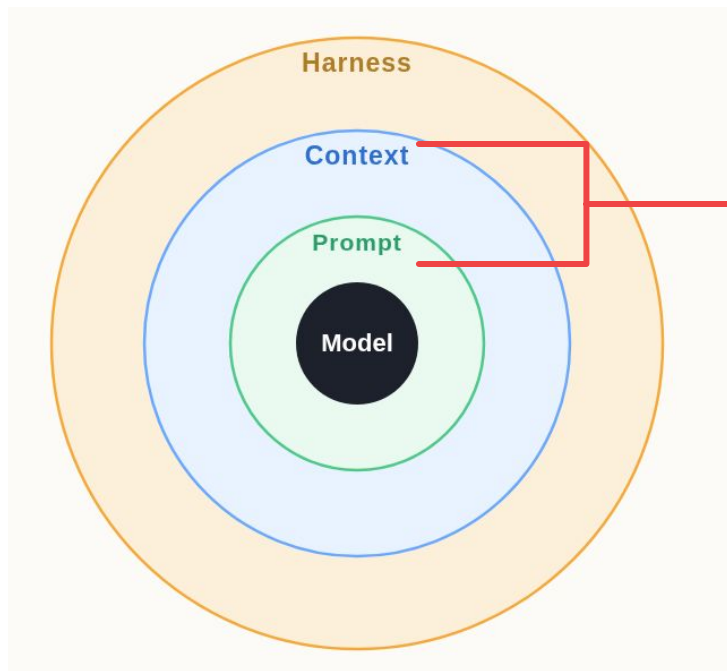
- Structured planning before implementation **improved coding success by up to 26.7%**, reducing failed generations and repeated implementation attempts. [Jiang et al.,\(2023\)](#)
- Removing the blueprint-planning agent from a structured coding pipeline **reduced accuracy by 14.8 percentage points**. [Mao et al, \(2025\)](#)
- Runs on the same coding task **varied by up to 30× in total token usage**, and higher token consumption did not produce greater accuracy. [Bai et al. \(2026\)](#)

How to make a plan?

How to make a plan?

Slides: go.wisc.edu/665254

Ans: Add context



Examples of context

- [plan.md](#)
- Skeleton code
- Github/Jira Story
- future-work.md
- Rules.md files
- Coding standards files
- Previous plans/context

Reading [Using PLANS.md for multi-hour problem solving by Aaron Friel](#)

Planning Demos

Feature-based dev & good prompting

Guide agentic coding with clear instructions

Context Engineering

defines overall guidelines for generating code, and what resources the agent can access to answer prompts.

Using .md files can help specify code standards.

Prompt Engineering

phrases your specific request to get the best possible result.

Guide agentic coding with clear instructions

BAD prompt:

Fix this spaghetti code.

Better prompt:

Update this SQL to use CTEs and window functions to improve performance and legibility. Retain all existing in-line comments. Add comments to explain functionality for each step.

Underspecified does not mean random

- A vague prompt fills every gap with the most statistically typical choice from its training data
- Your data/code is rarely the average case. Your weird instrument artifact is not in there
- So the failure mode is **plausible-but-wrong** oftentimes

Exercise: Prompt Engineering

```
# 1. Import modules and dataset
import pandas as pd
from sklearn import datasets
cancer = datasets.load_breast_cancer()

# 2. Give a bad/better-style prompt.

# 3. Evaluate and discuss results.
```

Break. Back at 6:35

Verification & Testing

What the research shows about checking

- In research code, checking agent output is the bottleneck
[Agents could not judge scientific validity. OpenAI field report, 2026](#)
- Agents multiply code written far more than code shipped
[240% more commits, 30% more releases. Demirer et al. 2026](#)

There is no escaping good data science

- **Know your data.** Distributions, outliers, units, missingness, class balance
- **Compare to a source of truth.** A published baseline, a hand-computed subset, a control
- **Ask a colleague.** Does this number seem plausible to you?
- **Know what your model is telling you.** If the top predictor makes no sense, that is a finding about your pipeline
- None of this moved to the agent. Only the typing did

Exercise. What is wrong with this? (5 min)

```
X, y = datasets.load_breast_cancer(return_X_y= True)

# Standardize and select the 10 best features
X = StandardScaler().fit_transform(X)
X = SelectKBest(f_classif, k=10).fit_transform(X, y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LogisticRegression(max_iter=5000).fit(X_train, y_train)
print(f"Test accuracy: {model.score(X_test, y_test):.3f}" )
```

Cheap checks that catch expensive mistakes

- **Shuffle the labels and re-run.** A sound pipeline collapses to chance. If it does not, something leaks
- **Print the baseline.** 94% stops being impressive when the majority class is 92%
- **Assert no sample is in both splits.** Repeated measures and tiles from one image belong on one side
- **Hunt duplicates before splitting.** Leakage with no code bug at all
- **Vary the seed.** A metric that swings across seeds is a variance problem, not a result

Hunt for decisions you did not explicitly make

Check assumptions at every step! Add tests for each.

- **Silently dropped rows.** dropna, inner joins, type coercions. Demand counts before and after
- **Defaults treated as decisions.** Every default it accepted is now a choice you own
- **Metric switcheroos.** You asked about accuracy, the writeup features F1
- **Suppressed problems.** Warnings silenced, try/except pass, the check deleted
- One prompt that pays for itself: **summarize every choice you made that I did not specify, and flag the risky ones**

Slides: go.wisc.edu/665254

Exercise. Test as contract (10 min)

```
# Prompt: Create analysis.py with load_data() and build_pipeline()
# so that pytest test_analysis.py passes. Run the tests to confirm.

def test_no_leakage_in_pipeline():
    from analysis import build_pipeline, load_data
    from sklearn.model_selection import cross_val_score
    X, y = load_data()
    pipe = build_pipeline()
    names = [n for n, _ in pipe.steps]
    assert "scaler" in names and "select" in names
    assert cross_val_score(pipe, X, y, cv=5).mean() > 0.9
```

MCP Tools & Skills

MCP vs API

Slides: go.wisc.edu/665254

Slides: go.wisc.edu/665254

Slides: go.wisc.edu/665254

Slides: go.wisc.edu/665254

Slides: go.wisc.edu/665254

Broader Implications: energy

How much energy do these models use?

- One chat query, about 0.3 Wh. An LED bulb burns that in two minutes
- One agentic coding session, about 41 Wh. Roughly 130 chat queries
[Simon Couch measured his own Claude Code sessions, 2026](#)
- A heavy day of agentic work, about a dishwasher run or a fridge for a day
[Simon Willison's own daily usage, 2026](#)
- Estimates vary a lot. No provider publishes per-query figures
[Hausfather's median session came out ~600 Wh, 2026](#)
- The water framing is mostly wrong, but aggregate demand is the real issue
[Inference, not training, is where most AI energy now goes](#)

Techniques to reduce token usage

- Pay attention to usage. (**/cost**)
- Police what model you are using and its level of difficulty. Make sure to switch.
- Minimize context used in prompts. (**/compact**, etc)
- Use sparingly.
- Use skills (graphify, caveman for ex.) to reduce input/output context.

Broader Implications:
is programming dead?

Is programming dead? The evidence says no

- Coding speed is only bottleneck. Advanced analyses require expert review and verification.
- Learning with AI can cost understanding
17% lower quiz scores, biggest gap in debugging. Anthropic 2026